

Student Meal Planner

Analysis Document

An Analysis Document for the Grubs4Scrubs Project

Date: May 2026

Author: Yao Cheng Zhou

Student Meal Planner

Analysis Document

An Analysis Document for the Grubs4Scrubs Project

Date: May 2026

Author: Yao Cheng Zhou

Table of Contents

Introduction 3

Project Description 3

Project Analysis 3

Gap 1: Budget Awareness 3

Gap 2: Student-Friendly Complexity 3

Gap 3: Meal Planning Integration 4

Gap 4: Nutritional Transparency 4

Existing Solutions 4

Stakeholder Analysis 4

Functional Requirements 6

User Management 6

Recipe Management 6

Meal Planning 6

Grocery List 6

Non-Functional Requirements 8

Domain Model 9

Implemented Entities 9

Planned Entities 9

Entity Relationships 9

Entity Relationship Diagram 10

Constraints 11

Assumptions 11

Conclusion 11

Introduction

This document covers the full analysis behind Grubs4Scrubs, a web app I'm building for my Semester 2 individual project at Fontys ICT. The whole idea is pretty straightforward: students need an easy way to find cheap, quick meals they can actually make in a dorm or small kitchen. I've been there, and most recipe apps don't really get what it's like cooking on a tight budget with barely any gear.

The purpose of this analysis is to break down the problem, figure out what already exists, define what the app needs to do, and map out the data model that supports everything. This document started as groundwork before building, but I've been updating it as development progresses to keep it honest about what's actually implemented versus what's still planned.

Project Description

Grubs4Scrubs is a student meal planner web application. The frontend runs on React 19 with Vite and custom CSS (I originally used Tailwind but removed it in Sprint 3 to keep styling consistent across all pages). The backend is an ASP.NET Core Web API built on .NET 10 with a SQL Server database, using raw ADO.NET for all data access. Authentication runs on BCrypt for password hashing and JWT tokens for session management.

The target audience is pretty specific: college and university students who are living on their own for the first time, cooking with limited equipment, and watching every euro they spend on food. The app needs to feel fast and simple because nobody wants to fight with a complicated interface just to figure out what's for dinner.

Project Analysis

I looked at the problem from a few different angles to figure out what gaps exist and where Grubs4Scrubs can actually add value.

Gap 1: Budget Awareness

Most recipe platforms don't show you what a meal actually costs. They'll list ingredients but you're on your own figuring out the price. For students, that's a dealbreaker. If you've got 15 euros to last the week, you need to know upfront whether a recipe fits your budget. Grubs4Scrubs includes an estimated budget per recipe and filters by price range so you can plan around what you can actually afford.

Gap 2: Student-Friendly Complexity

A lot of recipe apps assume you've got a full kitchen with every tool imaginable. Students typically have a single pan, a pot, maybe a microwave. The recipes on Grubs4Scrubs are specifically tagged by prep time and cook time, and they're written for people who don't have sous vide machines or stand mixers. Quick, simple, doable with basic gear.

Gap 3: Meal Planning Integration

Finding a recipe is one thing, but planning a whole week of meals is another problem entirely. Most free apps either don't have meal planning at all or lock it behind a subscription. Grubs4Scrubs will let users drag recipes into a weekly planner and auto-generate a grocery list from the combined ingredients. No paywall, no premium tier.

Gap 4: Nutritional Transparency

Students tend to either eat too much junk food or accidentally under-eat because they're busy. Each recipe in Grubs4Scrubs shows calories, protein, fats, and carbs right on the card. It's not a full diet tracker, but it gives you enough info to make reasonable choices without needing a separate app for nutrition.

Existing Solutions

Before building anything, I looked at three popular apps that overlap with what Grubs4Scrubs is trying to do. Here's how they compare:

Feature	AllRecipes	Mealime	Grubs4Scrubs
Budget per recipe	No	No	Yes
Student-focused recipes	No	Partially	Yes
Meal planning	No	Yes (paid)	Yes (free)
Grocery list generation	Yes (basic)	Yes	Yes
Nutritional info	Yes	Partial	Yes
Prep/Cook time filters	Yes	Yes	Yes
Free to use	Yes (with ads)	Freemium	Yes

The main takeaway is that none of the existing options combine budget tracking, student-specific recipes, and free meal planning in one place. AllRecipes is massive but generic and ad-heavy. Mealime is slick but locks planning behind a paywall. Grubs4Scrubs fills the gap by keeping everything free, focused, and built around what students actually need.

Stakeholder Analysis

I identified the key stakeholders and what each of them cares about:

Stakeholder	Interest	Impact
Students (end users)	Finding cheap, easy meals. Planning their week without overthinking it.	Primary users. Their feedback shapes what features matter most.
Fontys ICT (school)	Seeing proper software engineering practices. Clean architecture, testing, documentation.	They grade the project. The technical decisions need to demonstrate learning outcomes.
Developer (me)	Building something real that works. Learning React, .NET, and SQL Server properly.	I make all the decisions and do all the work. My skill growth directly affects quality.

Functional Requirements

I broke the functional requirements down using MoSCoW prioritization. Here's what the app needs to do:

User Management

ID	Requirement	Priority	Status
FR-01	Users can register with email and password	Must	Done
FR-02	Users can log in and receive a JWT token	Must	Done (backend)
FR-03	Users can log in via Google OAuth	Should	Planned
FR-04	Users can view and edit their profile	Should	Planned

Recipe Management

ID	Requirement	Priority	Status
FR-05	Users can browse all available recipes	Must	Done
FR-06	Users can view recipe details including nutritional info	Must	Done
FR-07	Users can filter recipes by category, budget, and prep time	Must	Done (category filters working, budget/prep time planned)
FR-08	Users can search recipes by keyword	Must	Done
FR-09	Users can create and submit their own recipes	Should	Done
FR-10	Users can edit and delete their own recipes	Should	Done

Meal Planning

ID	Requirement	Priority	Status
FR-11	Users can create a weekly meal plan	Should	Planned
FR-12	Users can assign recipes to specific days and meal types	Should	Planned
FR-13	Users can view their meal plan for the current week	Should	Planned

Grocery List

ID	Requirement	Priority	Status
FR-14	System generates a grocery list from the weekly meal plan	Could	Partial (manual shopping list with CRUD exists)
FR-15	Users can check off items on the grocery list	Could	Done

Non-Functional Requirements

ID	Requirement	Category
NFR-01	Pages should load within 2 seconds on a standard connection	Performance
NFR-02	The app should work on mobile and desktop browsers	Usability
NFR-03	Passwords must be hashed using BCrypt before storage	Security
NFR-04	The API should follow RESTful conventions	Maintainability
NFR-05	The codebase should use layered architecture with clear separation of concerns	Maintainability
NFR-06	Business logic should be covered by unit tests using xUnit with fake repositories	Quality
NFR-07	User authentication should use JWT tokens with BCrypt password hashing	Security

Domain Model

The domain model describes the core entities in the system and how they relate to each other. Some of these are already implemented in code, while others are planned for upcoming sprints.

Implemented Entities

User is the central entity. Every user has an email, a hashed password (BCrypt), a username, and optionally a Google ID for OAuth login. Registration and login are fully working on the backend through the AuthController, which issues JWT tokens on successful login. The frontend has LoginPage and SignUpPage components that talk to these endpoints.

Recipe holds all the info about a single meal: title, description, prep and cook times, servings, an estimated budget, a category, tags, tips, and nutritional data (calories, protein, fats, carbs as integers). Ingredients and instructions are stored as JSON strings inside the Recipe table because the school doesn't allow an ORM, so I'm keeping things manageable with raw ADO.NET queries. Each recipe can optionally belong to a user via UserId. Full CRUD is implemented on both frontend and backend. The RecipeViewPage has an edit modal for updating all fields and a delete button, plus an "Add to Shopping List" button that pushes ingredients directly to the ShoppingItem API.

ShoppingItem (originally planned as GroceryItem) is now a standalone entity with its own table, full backend CRUD (ShoppingItemController, ShoppingItemService, ShoppingItemRepository), and a working frontend page. Each item has a name, quantity (as a string), price, a checkbox for marking it off, and a UserId for future per-user filtering. The ShoppingListPage supports adding, editing, deleting, and checking off items. The current implementation is a manual shopping list (plus the recipe integration mentioned above) rather than auto-generated from meal plans, but the table and API are in place for that integration later.

Planned Entities

MealPlan represents a weekly plan created by a user. It has a name, a start date, and an end date. The frontend already has a MealPlannerPage with hardcoded sample data, but the backend tables and API don't exist yet.

MealSlot ties a specific recipe to a day of the week and a meal type (breakfast, lunch, dinner) within a meal plan. Each MealPlan will contain multiple MealSlots.

Favorite is a simple join table that lets users bookmark recipes they want to come back to. It just links a UserId to a RecipeId with a timestamp.

Entity Relationships

Here's how everything connects: a User can have zero or many Recipes (one-to-many). A User can also have zero or many ShoppingItems, MealPlans, and Favorites (all one-to-many). Each MealPlan contains one or many MealSlots (one-to-many), and each

MealSlot references exactly one Recipe (many-to-one). A Favorite links back to exactly one Recipe (many-to-one). On the frontend, recipes can push their ingredients directly into ShoppingItems through the "Add to Shopping List" button, which creates one ShoppingItem per ingredient.

Entity Relationship Diagram

The ERD below shows all entities with their attributes, data types, primary keys, and foreign keys. Blue entities (User, Recipe, ShoppingItem) are implemented in the codebase with full CRUD operations. The auth system (register, login, JWT) is also complete on the backend. Purple entities (MealPlan, MealSlot, Favorite) are planned for future sprints.

Entity Relationship Diagram for Grubs4Scrubs showing implemented and planned entities. The diagram uses color coding to distinguish between implemented and planned entities. Cardinality labels (1, 0..*, 1..*) show how many instances of one entity can relate to another. Note that Ingredients and Instructions are currently stored as JSON strings inside the Recipe table rather than as separate normalized tables.

Constraints

There are a few constraints that shape how I can build this project:

- No ORM allowed. The school requires raw ADO.NET with parameterized SQL queries, which means I'm writing all the data access code by hand. It's more work, but it forces me to actually understand what's happening at the database level.
- Single developer. I'm the only person working on this, so I have to be realistic about scope. That's why the MoSCoW prioritization matters: Must-haves get done first, everything else is stretch.
- Time-boxed to one semester. The project runs alongside other coursework, so I can't spend unlimited time on it. Sprint planning and backlog management keep things on track.
- SQL Server as the database. This is a school requirement. I'm running it locally for development.
- The frontend must be a JavaScript framework. I chose React because it's the most widely used and has the best job market relevance.

Assumptions

A few assumptions I'm working with:

- Users have a modern browser (Chrome, Firefox, Safari, Edge). I'm not supporting Internet Explorer.
- Users have basic internet access. The app isn't designed for offline use right now.
- Recipe nutritional data is estimated and provided by the recipe creator. It's not pulled from a certified nutrition database.
- The app will be hosted locally during development. Deployment to a cloud provider is a stretch goal.

Conclusion

This analysis has held up well through actual development. The gaps I identified early on (budget awareness, student-friendly recipes, free meal planning, nutritional transparency) still drive the features I'm building. Most of the Must-have requirements are done: users can browse recipes, view full details with nutrition info, search and filter, create new recipes, and edit or delete existing ones. The auth system (register, login, JWT) is working on the backend. The shopping list is fully functional with its own CRUD API and frontend page, including the ability to push ingredients straight from a recipe.

What's left is mostly the Should-have and Could-have features: meal planning with the weekly drag-and-drop planner, Google OAuth, user profiles, and auto-generating grocery lists from meal plans. The domain model and ERD have proven accurate. ShoppingItem moved from planned to implemented without needing schema changes, which tells me the upfront design work was worth it. Unit tests cover the core RecipeService logic with a FakeRepository, and the layered architecture keeps things organized even as the codebase grows. The foundation is solid, and the remaining features are well-scoped for the time I have left.